



Inteligência Artificial para Jogos I

Aulas 9 e 10: Representação do Mundo

Prof. Charles Madeira

Objetivos da aula

- Apresentar o modelo de representação do ambiente nos jogos digitais, como elemento fundamental para a tarefa de percepção do estado do mundo simulado pela IA que joga jogos
- Introduzir formas de abstração do mundo, que podem ser apresentadas no formato de estruturas de dados
- Introduzir árvores e grafos como formas de organização dos dados e otimização do processo de busca de elementos no mundo do jogos digitais

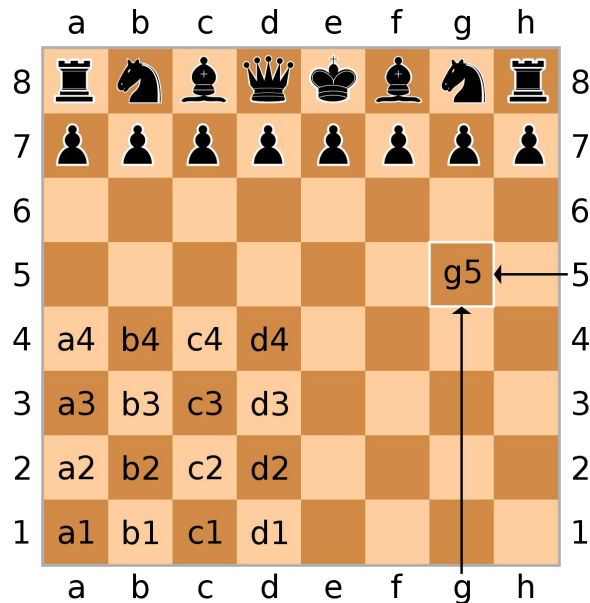


Definição de modelo

“Um modelo é uma simplificação da realidade”

Nem tudo precisa estar presente em um modelo

O modelo do mundo de um jogo é uma simplificação, com **informações suficientes para implementar as mecânicas do jogo, suas regras e permitir a imersão do jogador**



Estado do mundo

Um mundo possui um **estado**, representado por seu modelo

As tomadas de decisão são sobre o estado que ele se encontra

Como representar um estado do mundo e como ele se modifica?

Age of Empires IV

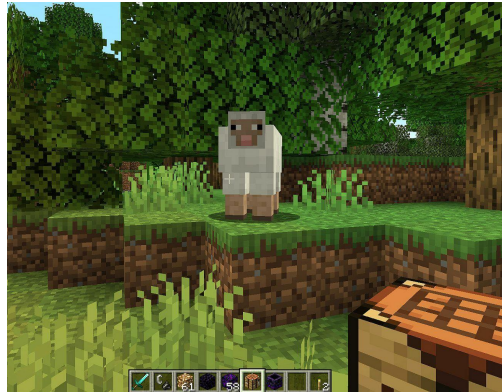


Formas de representar o mundo

Dependendo das características (mecânica, regras...) do jogo, há jogos:

- Sem representação alguma (ex: karaokê 😊)
- Com representações sem abstrações (ex: alguns jogos de point-&-click)
- Em que há elementos do jogo (ex: personagens) “posicionados”

Campo minado



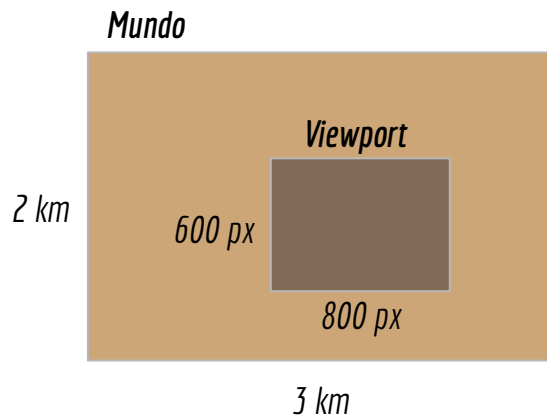
Minecraft

Posição absoluta e relativa

⚠ Estamos abordando aqui a representação do mundo útil para a IA do jogo... não para a visualização do jogo

O mundo pode ser visualizado de diferentes formas, e o que é útil para a IA não tem necessariamente a ver com o pixel ou o tile apresentado

A IA utiliza a **posição absoluta** da entidade no mundo virtual, enquanto a visualização utiliza a **posição da entidade relativa à área projetada na tela (viewport)**



Diferentes formas

Cada jogo é único e pode combinar as abstrações e formas de organizar existentes, bem como definir sua própria forma

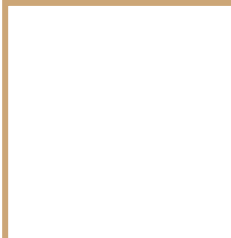
Godus



Como o estado do mundo é alterado?

Um jogo é uma “simulação interativa” e muitos modelos de simulação podem ser usados:

- Simulação discreta
 - Mudança do estado do mundo ocorre em intervalos fixos de tempo
 - Jogos baseados em turno
 - Jogos em tempo “real” (paralelismo ou pseudo-paralelismo)
- Simulação a eventos discretos
 - Mudança do estado do mundo ocorre apenas em decorrência de eventos (ex: daqui a 10s a bomba explode)
 - Útil em jogos com um mundo vasto, quando é impraticável simular o que pode ocorrer em todas as entidades a cada iteração



Malhas regulares



Abstrações do mundo

Usando malhas (grids) regulares

- Malhas quadradas/cúbicas
 - Proj. ortogonais ou isométricas são formas de visualizar
- Malhas hexagonais

Age of Empires (proj. isométrica)



Terraria (proj. ortogonal)



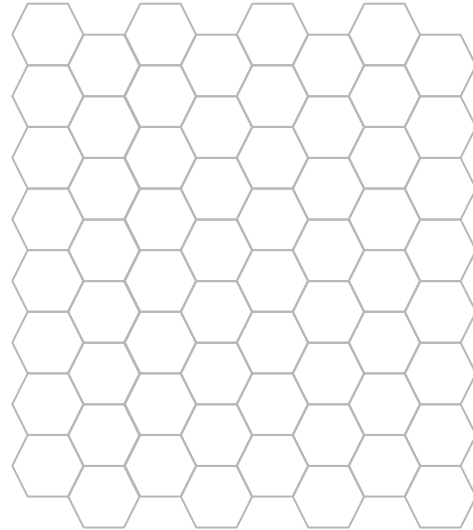
Civilization (malha hexagonal)



Abstrações de mundos usando malhas regulares

Forma mais simples de simplificar um mundo virtual

Mundo discretizado em intervalos com tamanho e forma pré-definidos



Utilidade das malhas regulares para a IA

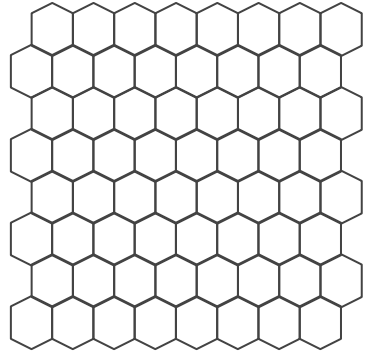
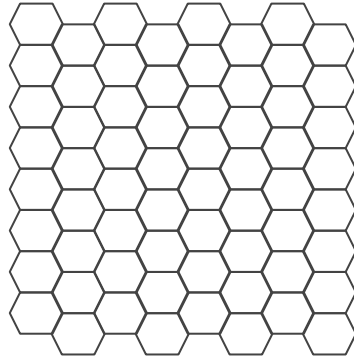
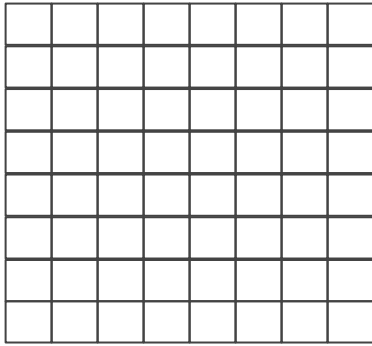
Nas tomadas de decisão da IA, em geral, é necessário saber o que está próximo sem precisar percorrer todas as entidades do jogo

1. Tem algum recurso útil por perto?
2. Tem algum inimigo próximo?
3. Quais são os obstáculos para se deslocar a um determinado ponto?
4. O que é preciso atualizar a cada iteração do jogo?

As subdivisões (células da malha) ajudam a limitar o escopo de percepção da IA

Quando usar...

- nenhuma malha (posição absoluta)?
- malha quadrada (square-grid)?
- malha hexagonal (hexa-grid)?

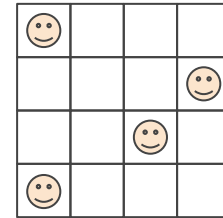


Variações no posicionamento

As entidades precisam então estar posicionadas em UMA ÚNICA célula da malha

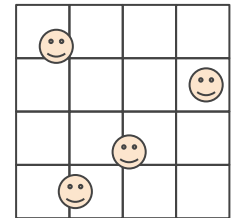
- Posicionamento fixo (no centro da forma)?

- Posição da entidade é definida pelas coordenadas da célula
- Ótimo para testar colisões



- Posicionamento livre (dentro da forma)?

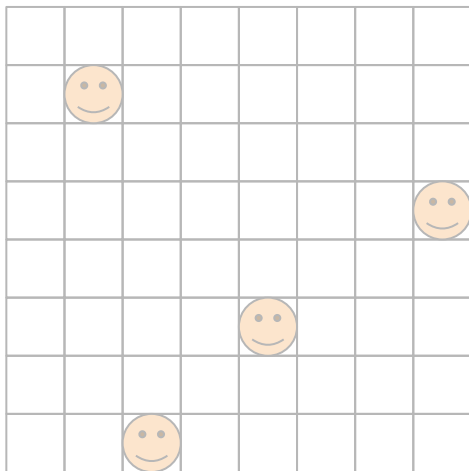
- Posicionamento é definido pelas coordenadas da célula e pelo deslocamento dentro dela, ou
- Posicionamento “real”, mas o posicionamento da célula pode ser calculado



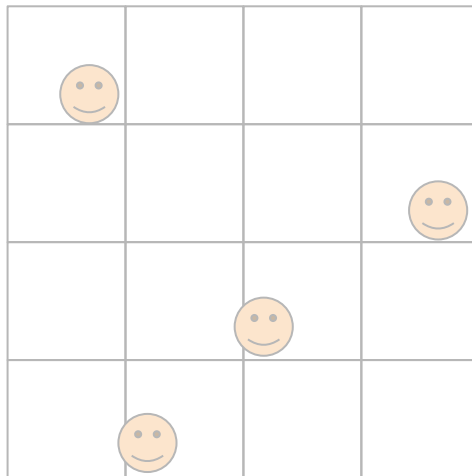
Tamanho das células

Qual o tamanho adequado para o meu jogo?

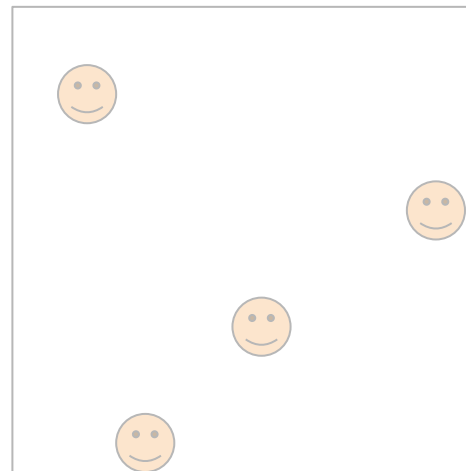
posição de um personagem



região próxima de um personagem



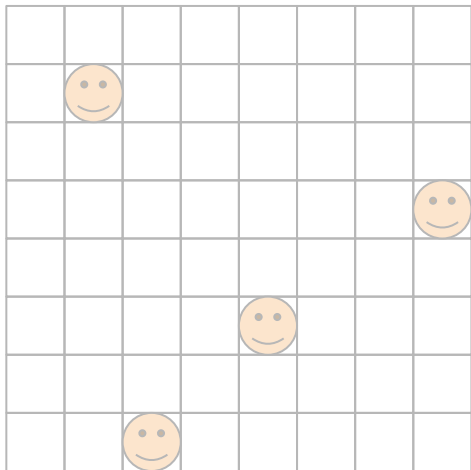
cidade ou nível contendo vários personagens



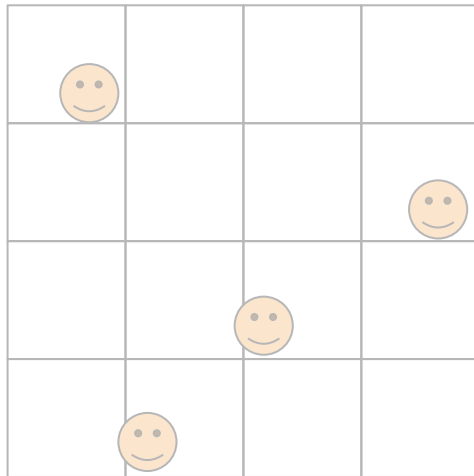
Organização dos elementos

Como estruturar os dados das células da malha?

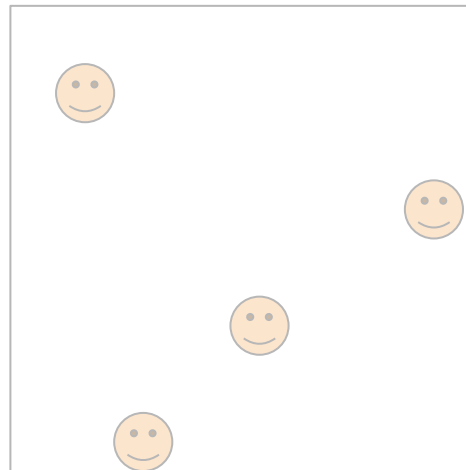
posição de um personagem



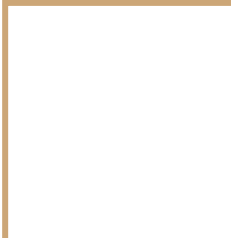
região próxima de um personagem



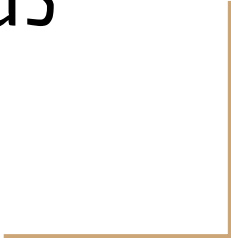
cidade ou nível contendo vários personagens



Necessário para otimizar o processo de busca por informação



Árvores e representações hierárquicas



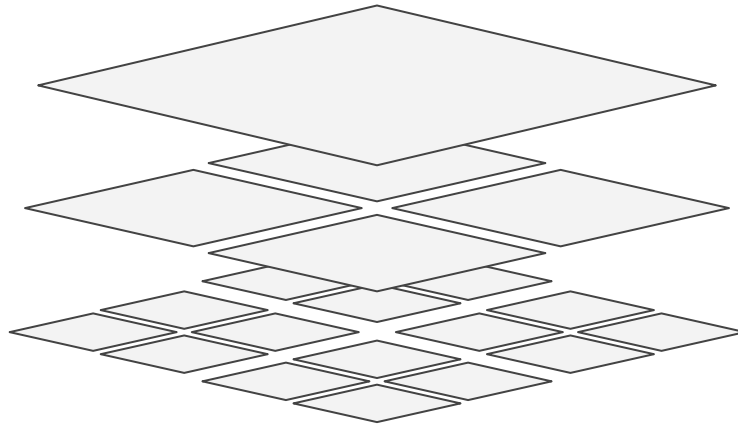
Abstrações em mundos

Muitas vezes, os jogos precisam “abstrair os detalhes” em função do foco (localização do jogador)



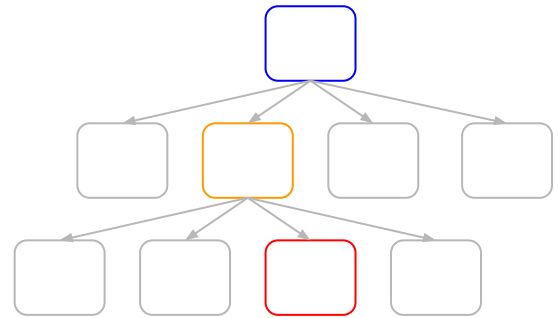
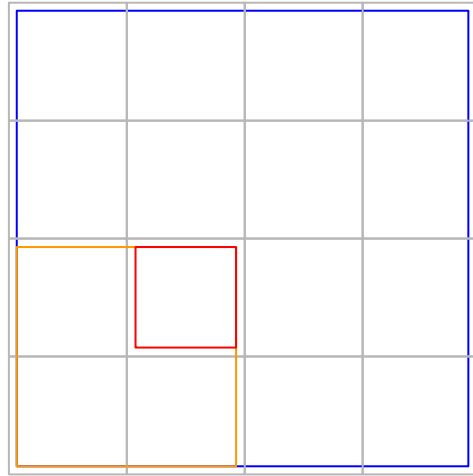
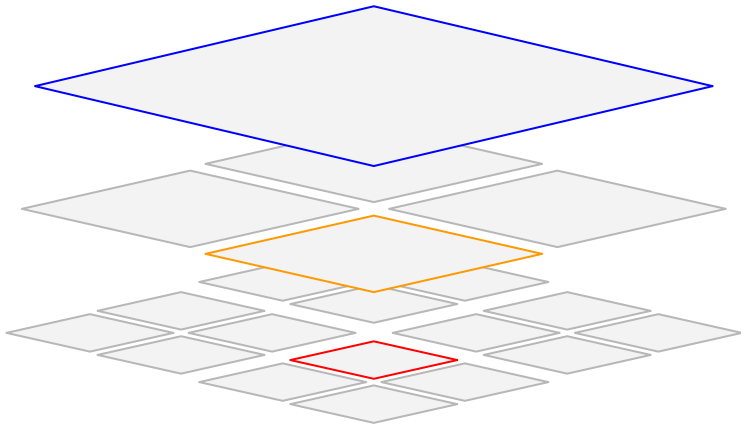
Abstrações hierárquicas

Forma estruturada de “particionar” o plano / espaço em diferentes camadas, cada uma com seu nível de abstração



Estruturas hierárquicas através de árvores

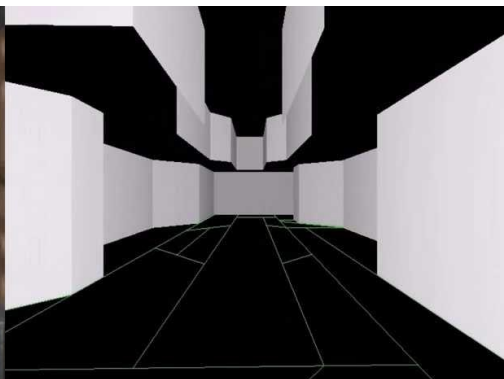
Árvore é uma estrutura hierárquica útil para organizar dados e otimizar algoritmos (busca, planejamento, localização de entidades, ...)



Organização das informações

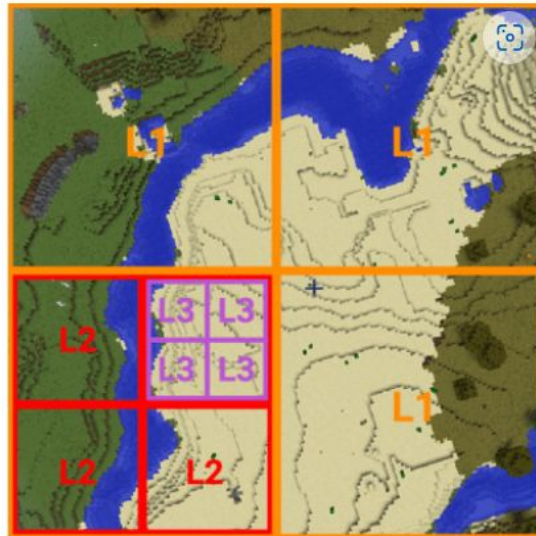
Usando representações hierárquicas:

- Quadrees/Octrees
- Binary Search Partition (BSP)
- Kd-tree



Doom (BSP)

Quadtree



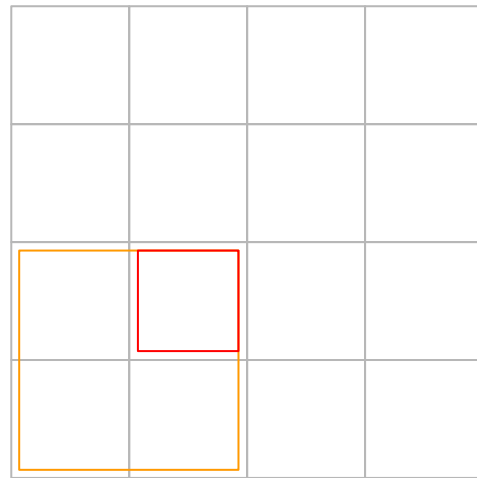
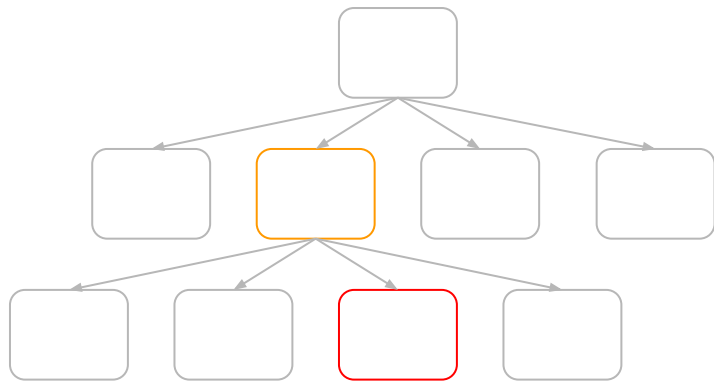
Quadrees (2D) e Octrees (3D)

Quadrees são estruturas de dados em árvore para particionar um espaço 2D em até quatro quadrantes

Octrees fazem o mesmo para um espaço 3D, dividindo-o em até oito octantes

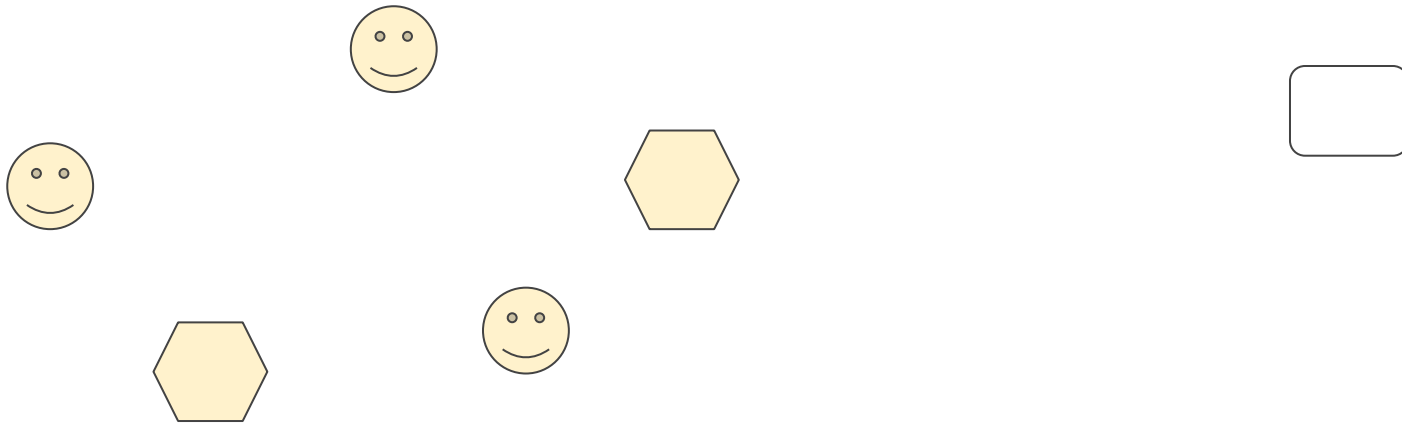
Subdivisão espacial hierárquica através de uma malha regular

Vantagens da estrutura regular, aliada a facilidade de busca no mundo



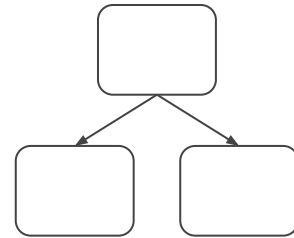
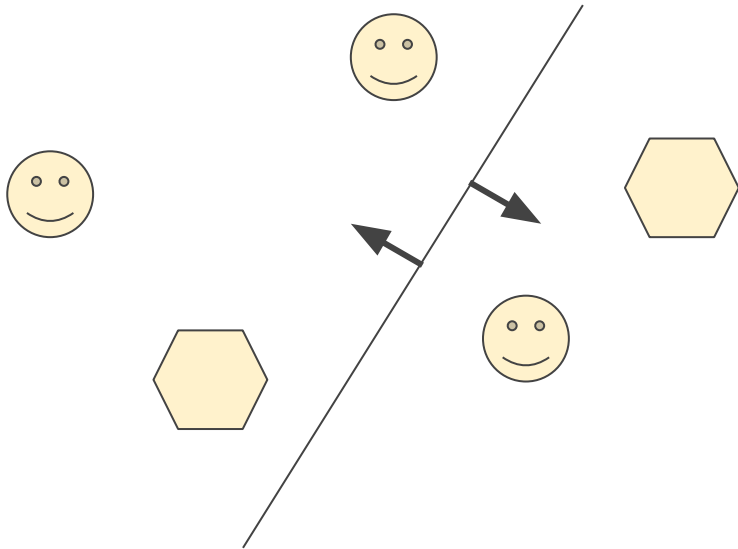
BSP - *Binary Space Partition*

Uso de uma árvore binária para subdividir um espaço em convexos definidos de hiperplanos
Divide sucessivas regiões a partir de uma reta (2D) ou plano (3D)



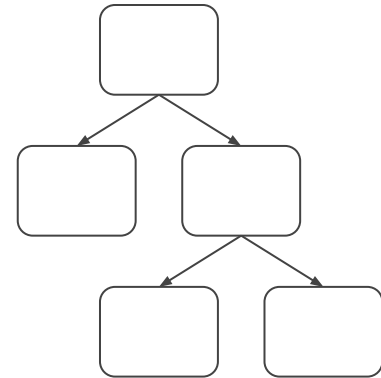
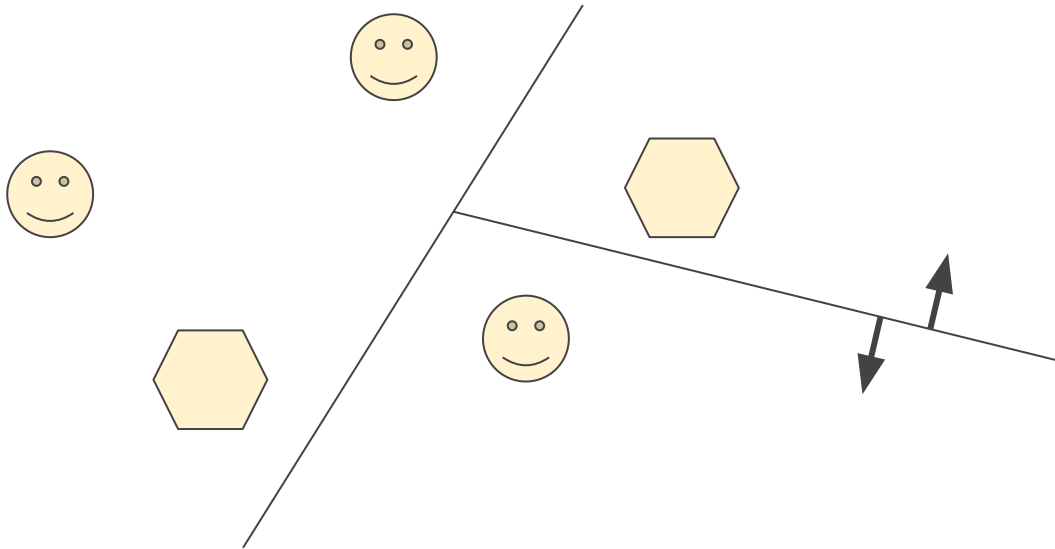
BSP - *Binary Space Partition*

Uso de uma árvore binária para subdividir um espaço em convexos definidos de hiperplanos
Divide sucessivas regiões a partir de uma reta (2D) ou plano (3D)



BSP - *Binary Space Partition*

Uso de uma árvore binária para subdividir um espaço em convexos definidos de hiperplanos
Divide sucessivas regiões a partir de uma reta (2D) ou plano (3D)



Utilização da BSP

BSP foi inicialmente utilizada para o processo de renderização (jogos 2D½)

Wolfenstein 3D (1992)



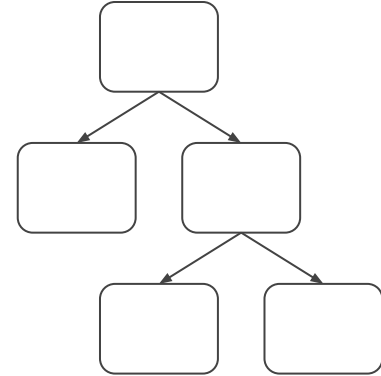
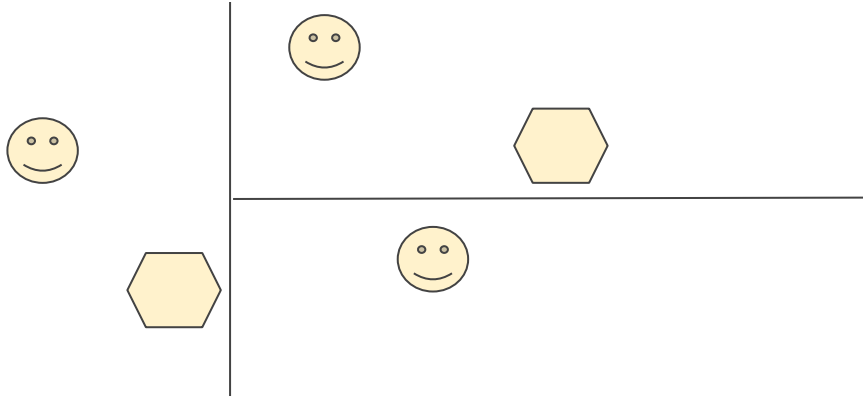
Doom (1993)



Kd-Tree

BSP é adequado quando a geometria do mundo é clara, tais como em cenas interiores
Para cenas exteriores, pode-se simplificar a representação

Kd-Tree é uma BSP na qual as divisões são ortogonais → comparações simplificadas



O que é uma “árvore”?

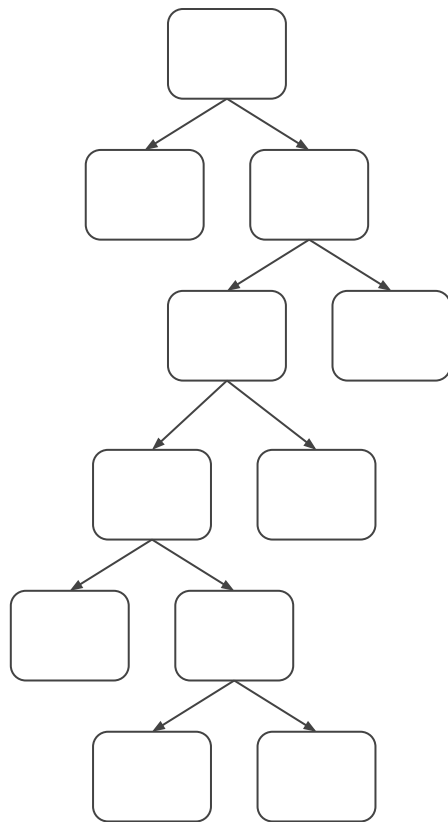
Tipo Abstrato de Dado (estrutura recursiva):

- Um nó pode possuir filhos
- Todo filho é também um nó

Quanto ao número máximo de filhos:

- Binária (até 2 filhos)
- Ternária (até 3 filhos)
- n-ária, ...

Normalmente, há critérios para escolher filhos em um processo de busca por informações (árvores de busca)



Algumas definições

- nó / raiz / nó-folha
- pai/filho, descendente / ancestral,...
- grau de um nó (quantidade de filhos)
- grau da árvore (quantidade máxima de filhos que um nó pode possuir)
- profundidade / nível de um nó (distância da raiz até o nó)
- altura de um nó (maior distância do nó para um nó-folha)
- altura da árvore (maior distância da raiz a um nó-folha)
- largura de um nível (quantos nós existem no nível)

Como implementar uma árvore?

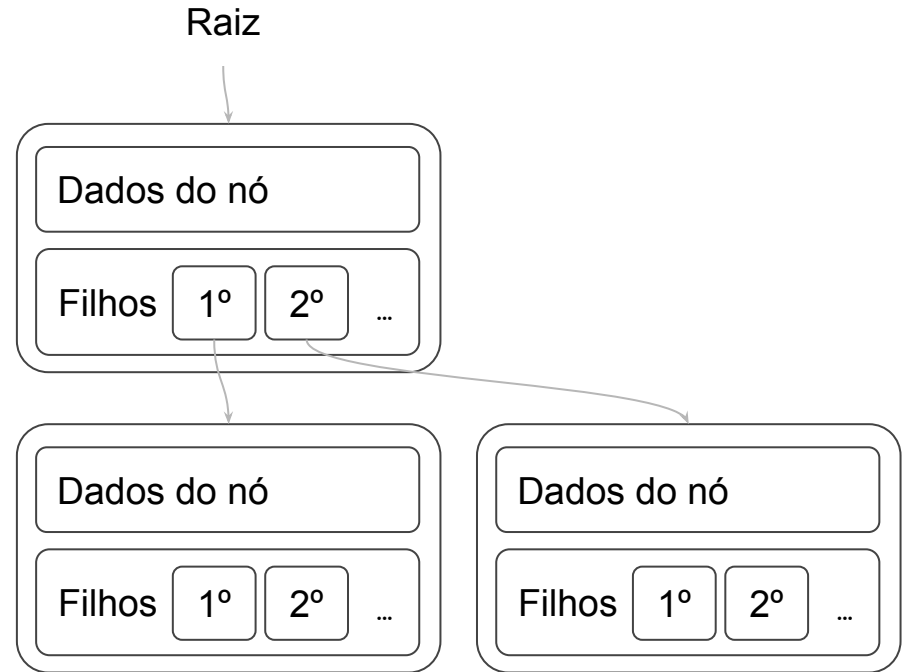
tipo Nó:

dados: qualquer

filhos: coleção de nós

tipo Árvore:

raiz: Nó



Exemplo: Quadtree

```
class QuadTreeNode {
private:
    int left, top, right, bottom; // coordenadas dos limites
    QuadTreeNode *children[4];
    vector<GameObject*> objects;

public:
    QuadTreeNode(int l, int t, int r, int b): left(l), top(t), right(r), bottom(b) {
        children[0] = children[1] = children[2] = children[3] = nullptr;
    }
    ~QuadTreeNode() {
        for (int i = 0; i < 4; i++) {
            if (children[i] != nullptr) delete children[i];
        }
    }
    bool isLeaf() const { return children[0] == nullptr; };
    ...
}
```

Exemplo: Quadtree

```
...
QuadtreeNode* getNW() const { return children[0]; };
QuadtreeNode* getNE() const { return children[1]; };
QuadtreeNode* getSW() const { return children[2]; };
QuadtreeNode* getSE() const { return children[3]; };

void divide() {
    if (!isLeaf()) return;
    int midX = (left + right) / 2;
    int midY = (top + bottom) / 2;
    children[0] = new QuadtreeNode(left, top, midX, midY);
    children[1] = new QuadtreeNode(midX, top, right, midY);
    children[2] = new QuadtreeNode(left, midY, midX, bottom);
    children[3] = new QuadtreeNode(midX, midY, right, bottom);
    // TODO distribuir os objetos nos filhos
}
};
```

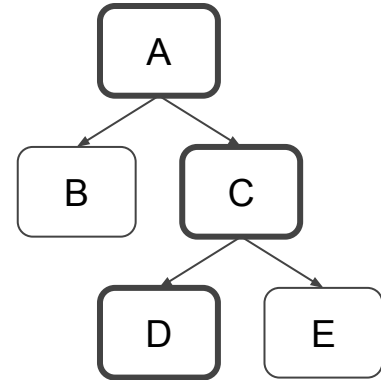
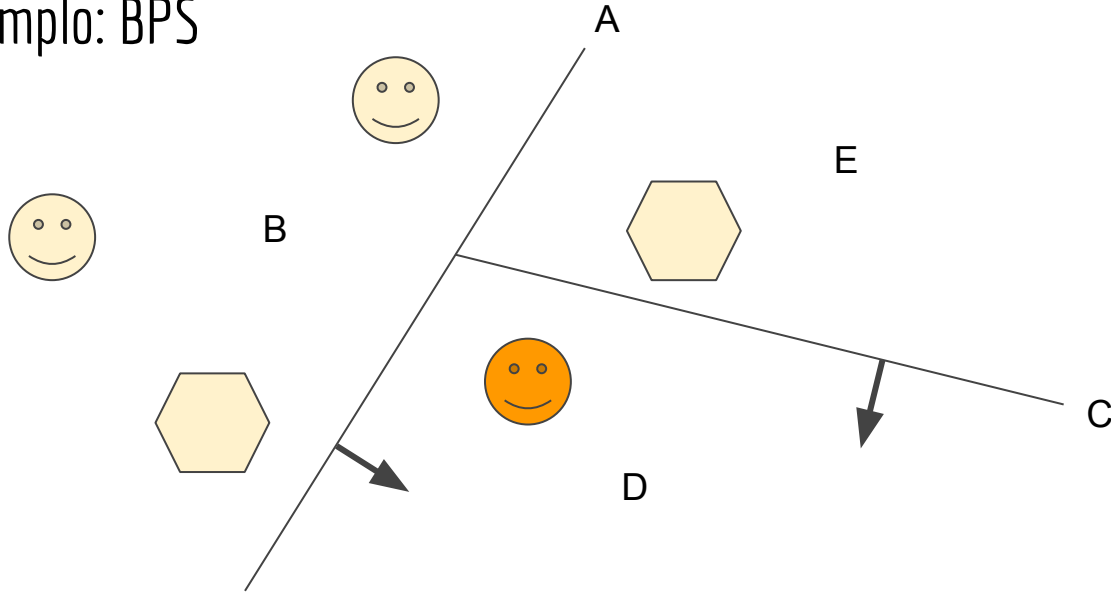
Exemplo: Quadtree

```
class Quadtree {  
    private:  
        QuadtreeNode *root;  
  
    public:  
        Quadtree(int width, int length) {  
            root = new QuadtreeNode(0, length, width, 0);  
        }  
        ~Quadtree() {  
            delete root;  
        }  
        ...  
};
```

Busca em árvores

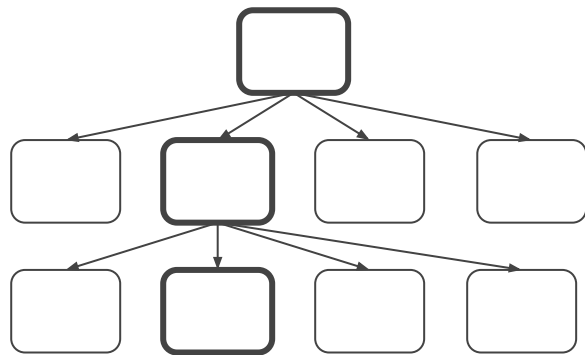
Árvores de busca são árvores que possuem um critério na escolha de onde buscar um filho

Exemplo: BPS



Exemplo: Busca na Quadtree

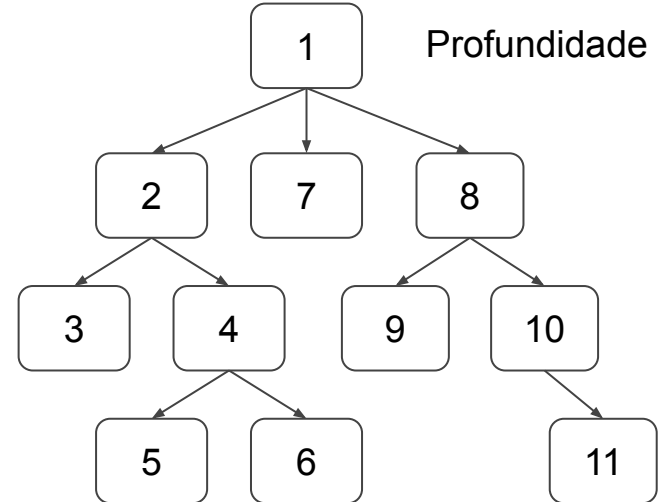
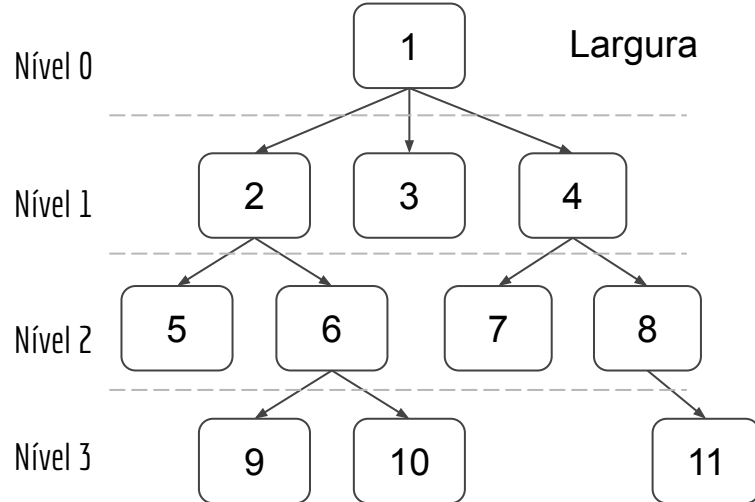
```
class QuadtreeNode {  
    ...  
    vector<GameObject*> nodeObjects() const {  
        return objects;  
    }  
    vector<GameObject*> getObjects(int x, int y) const {  
        if (isLeaf()) return objects;  
  
        int midX = (left + right) / 2;  
        int midY = (top + bottom) / 2;  
        QuadtreeNode *child = x < midX  
            ? (y < midY ? children[0] : children[2])  
            : (y < midY ? children[1] : children[3]);  
        return child->getObjects(x, y);  
    }  
    ...  
}
```



Percurso em árvores

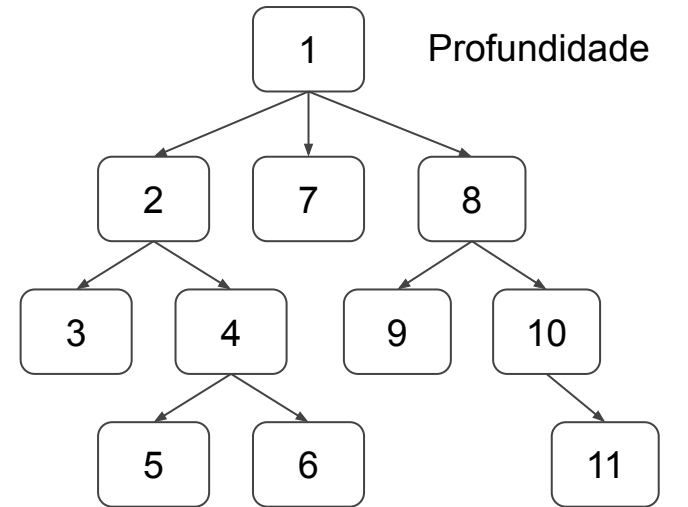
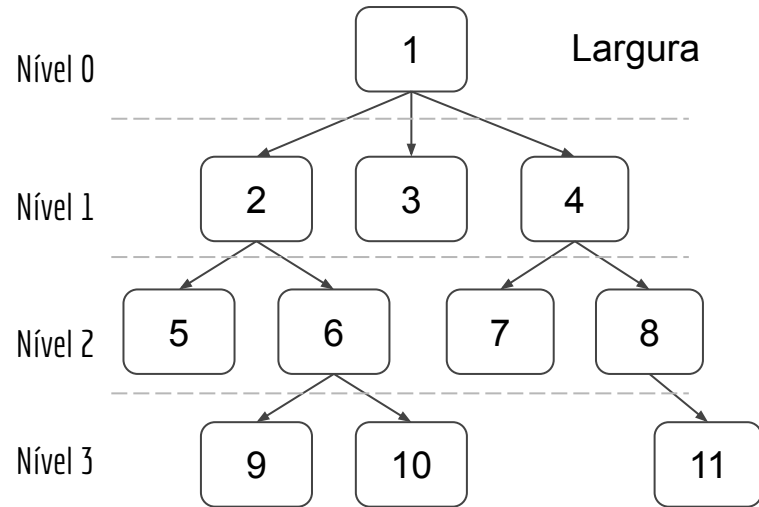
Define a ordem de visita dos nós em um processo de busca geral

- Largura: percorre todos os nós de um nível para ir depois para o nível seguinte
- Profundidade: percorre todos os nós de um caminho até um nó-folha



Percurso em largura ou em profundidade

Uso de uma estrutura de dados diferente para cada caso

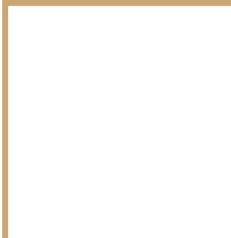


Exemplo: busca em largura em uma Quadtree

```
class Quadtree { ...
    QuadtreeNode* searchNode(string objID) {
        queue<QuadtreeNode*> q; // fila
        q.push(root);
        while (!q.empty()) {
            QuadtreeNode *node = q.front(); q.pop();
            if (node->isLeaf()) {
                for(auto obj: node->nodeObjs()) {
                    if (obj->getID() == objID) return node;
                }
            } else {
                q.push(node->getNW()); q.push(node->getNE());
                q.push(node->getSW()); q.push(node->getSE());
            }
        }
    }
    ...
}
```

Exemplo: busca em profundidade em uma Quadtree

```
class Quadtree { ...
    QuadtreeNode* searchNode(string objID) {
        stack<QuadtreeNode*> q; // pilha
        q.push(root);
        while (!q.empty()) {
            QuadtreeNode *node = q.front(); q.pop();
            if (node->isLeaf()) {
                for(auto obj: node->nodeObjs()) {
                    if (obj->getID() == objID) return node;
                }
            } else {
                q.push(node->getNW()); q.push(node->getNE());
                q.push(node->getSW()); q.push(node->getSE());
            }
        }
    }
    ...
}
```



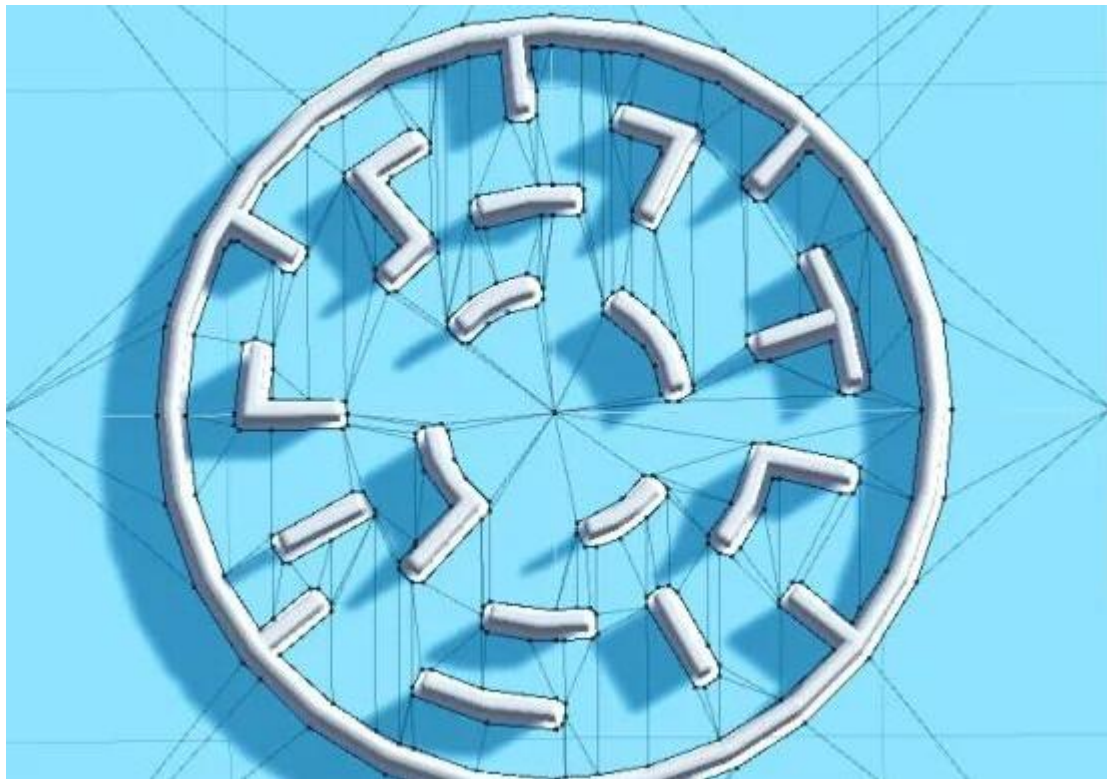
Grafos e subdivisões planares



Malhas não regulares

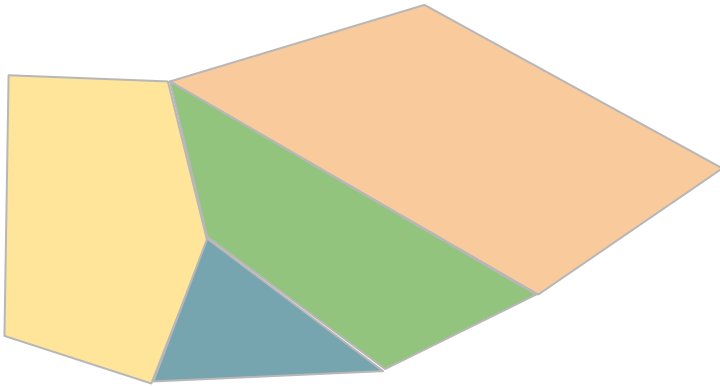
Muitas vezes, os jogos precisam dividir suas áreas em áreas menores, não regulares

- Navegação
- Representações geográficas distintas
- Análise do terreno
- ...



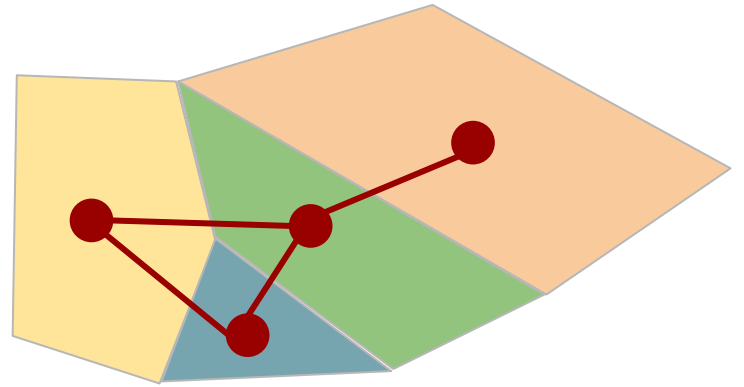
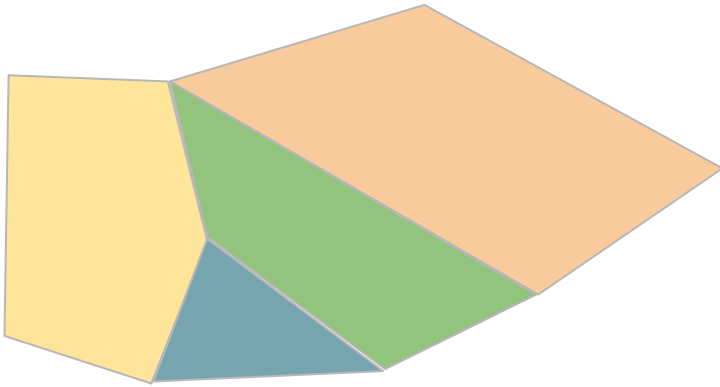
Malhas não regulares como subdivisões

1. Subdivisões representam áreas do ambiente
2. Áreas possuem conexões umas com as outras



Subdivisões como grafos

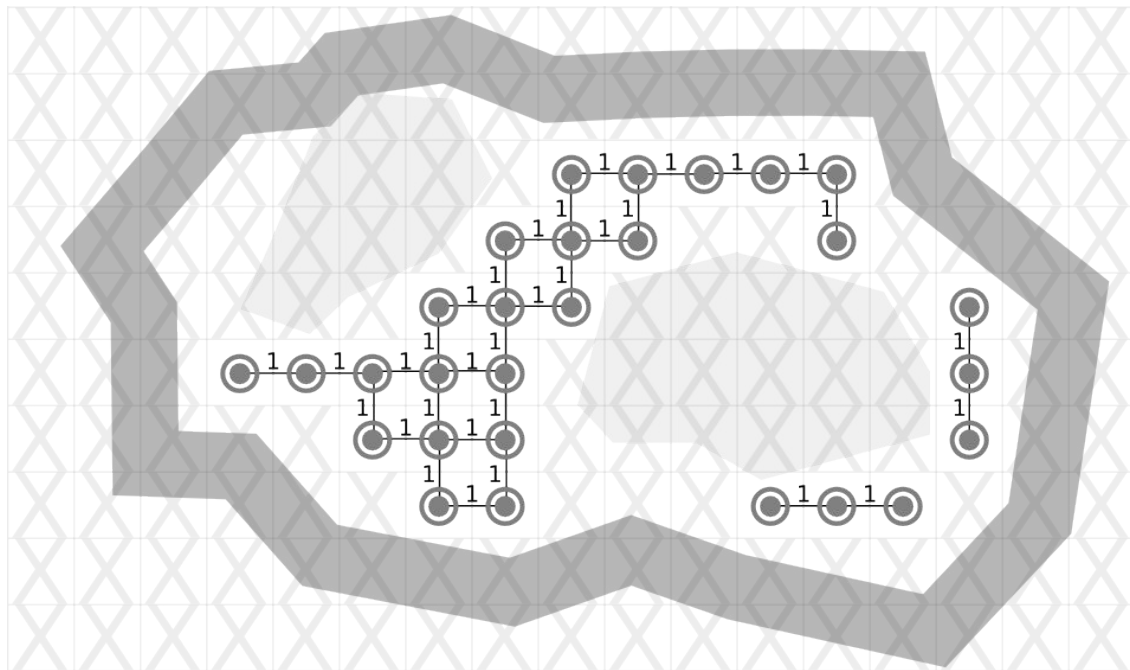
1. Cada região é representada por um ponto (nó) do grafo
2. Se uma região é vizinha de outra, então há uma ligação entre os nós correspondentes



Malhas (grids)

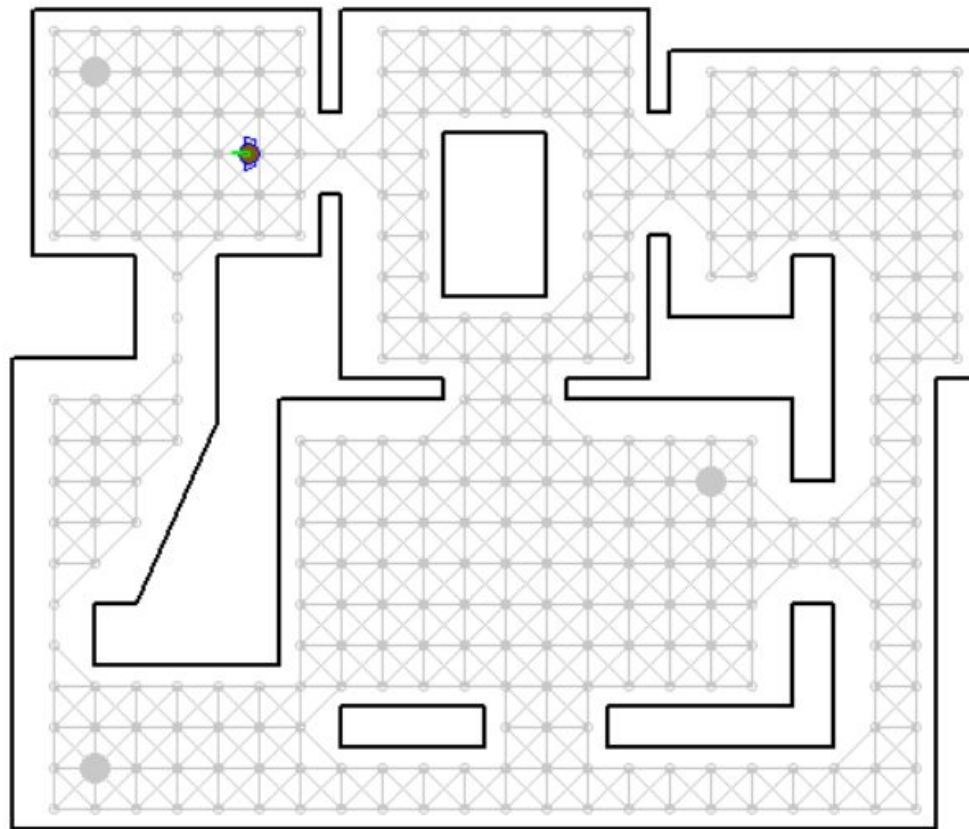
Forma eficiente de representar
ambientes pequenos ou
labirintos

Também é um grafo



Malhas (grids)

Grafos de navegação

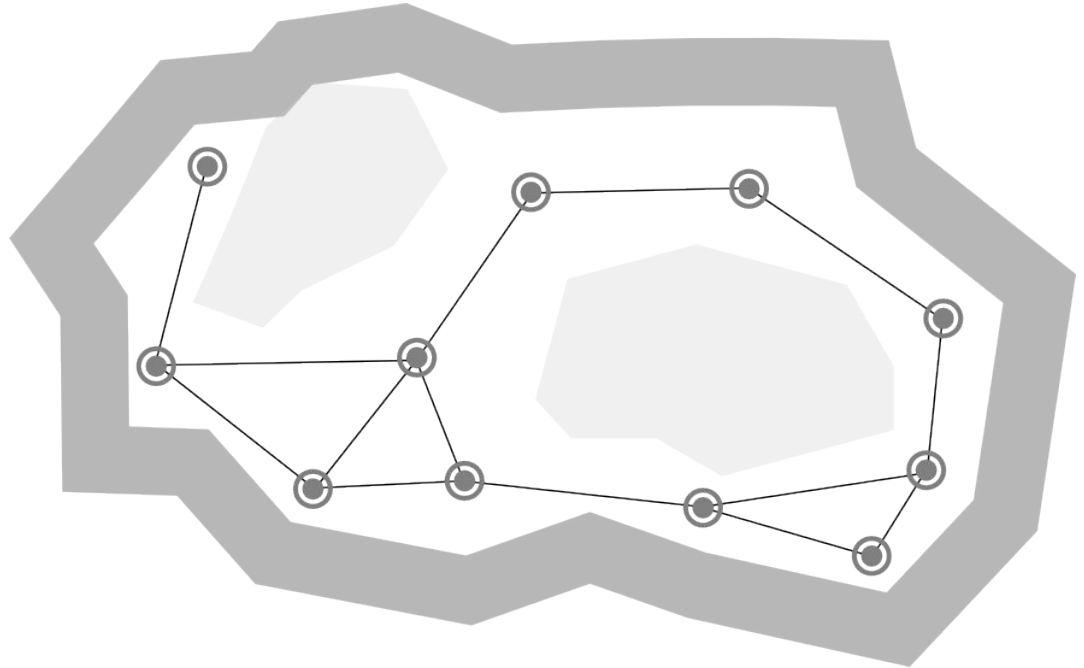


Waypoints

Pontos e caminhos do mundo em
que os personagens podem se
deslocar para evitar os obstáculos

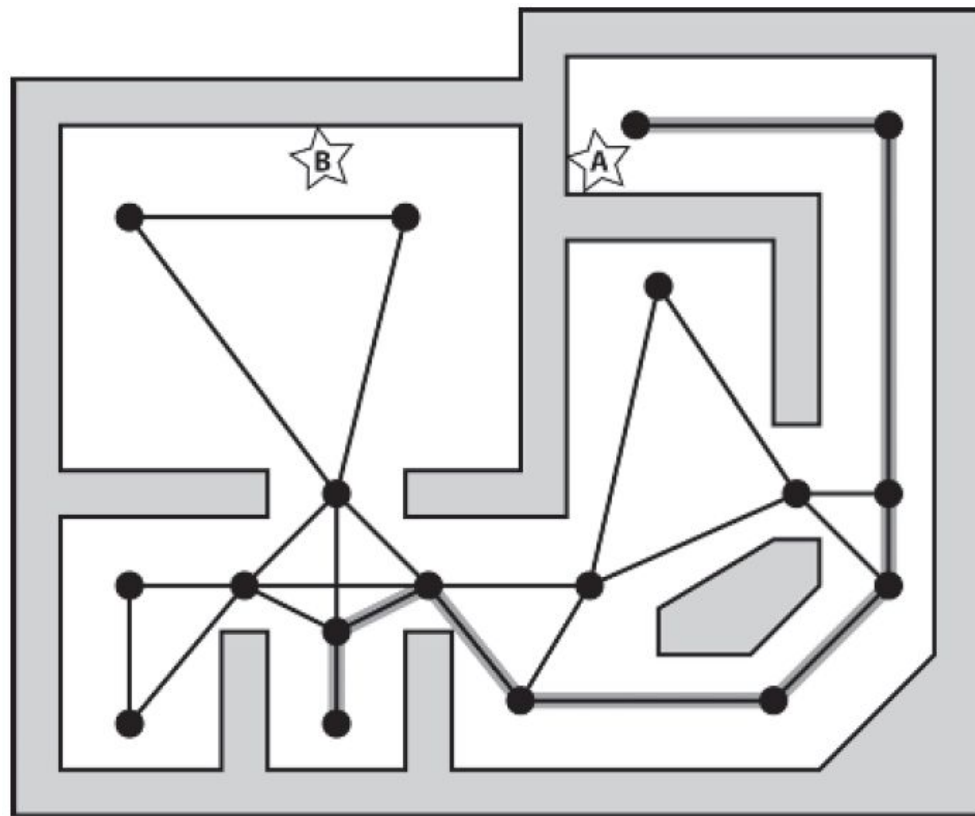
Um grafo denso é pouco eficiente

Um grafo esparso não é adequado para busca de caminhos



Waypoints

Grafos de navegação



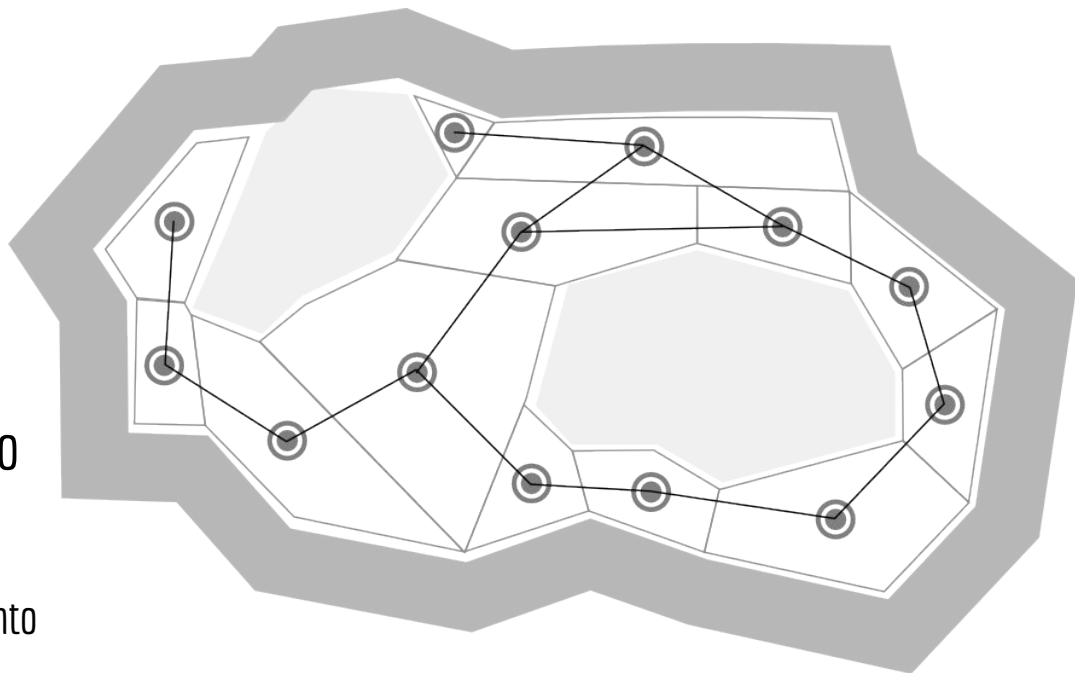
Malha de navegação - navigation mesh ou grafo de área

Forma eficiente de representar áreas
“navegáveis” do mundo

Polígonos convexos

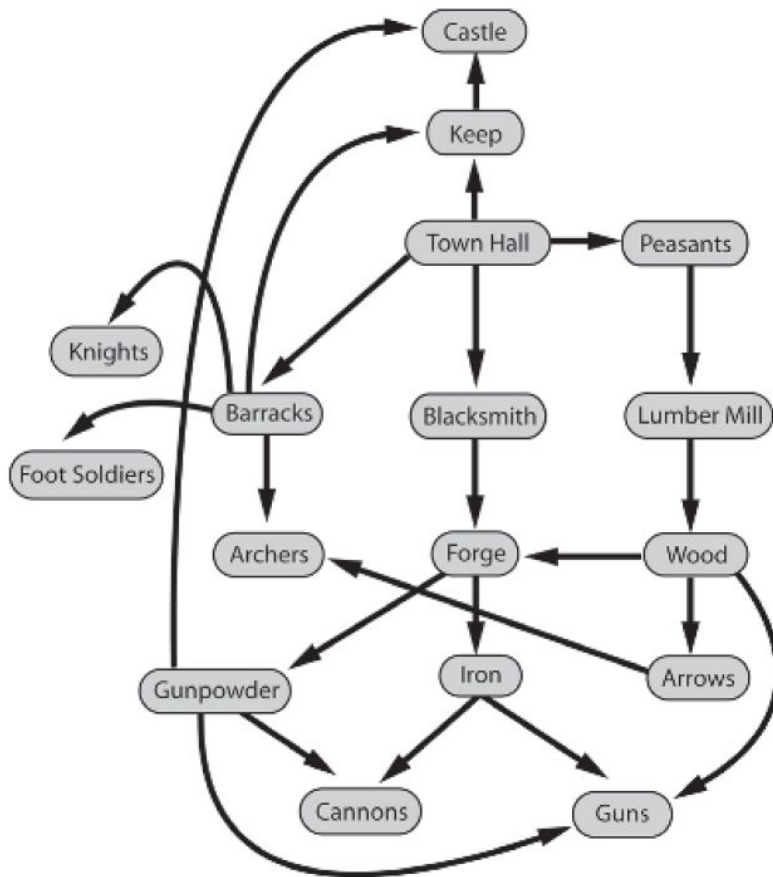
Personagens podem se deslocar em
uma célula associada a um nó do grafo

Malha ideal → quando um NPC pode ir de
qualquer ponto de uma região a qualquer ponto
de outra região adjacente sem colidir com
obstáculos



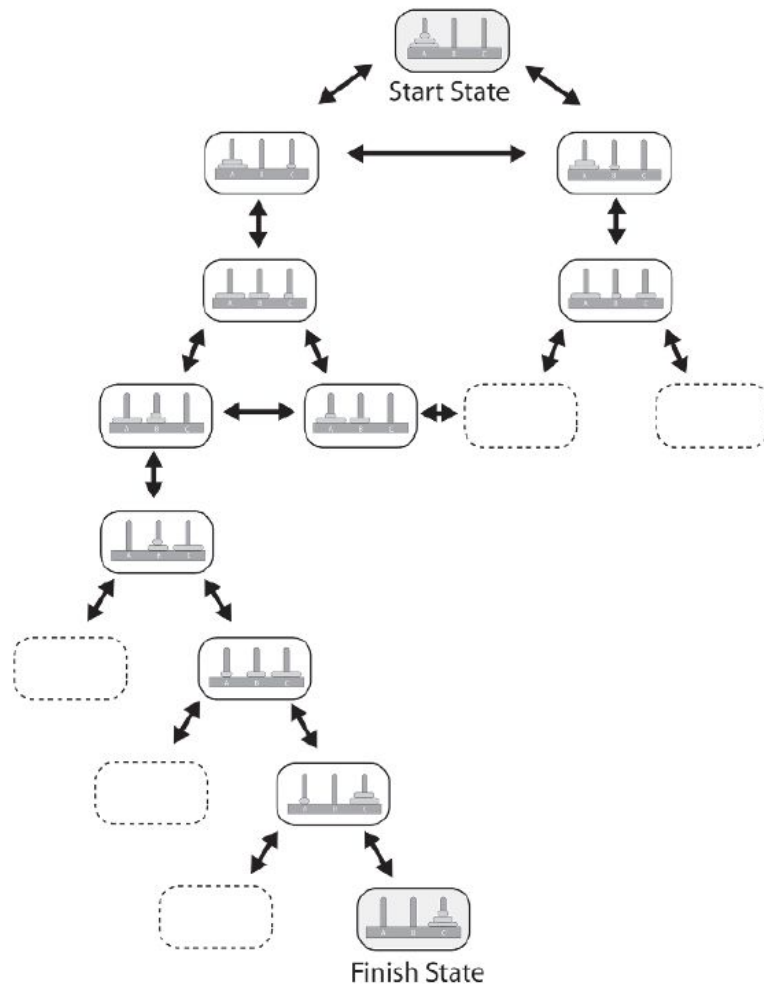
Grafos em jogos

Grafos de dependência



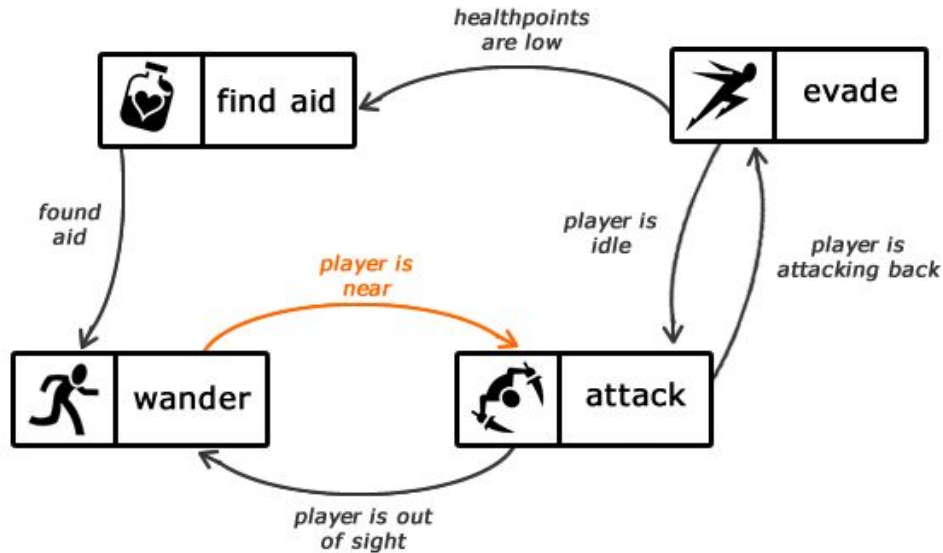
Grafos em jogos

Grafos de estados (planejamento)



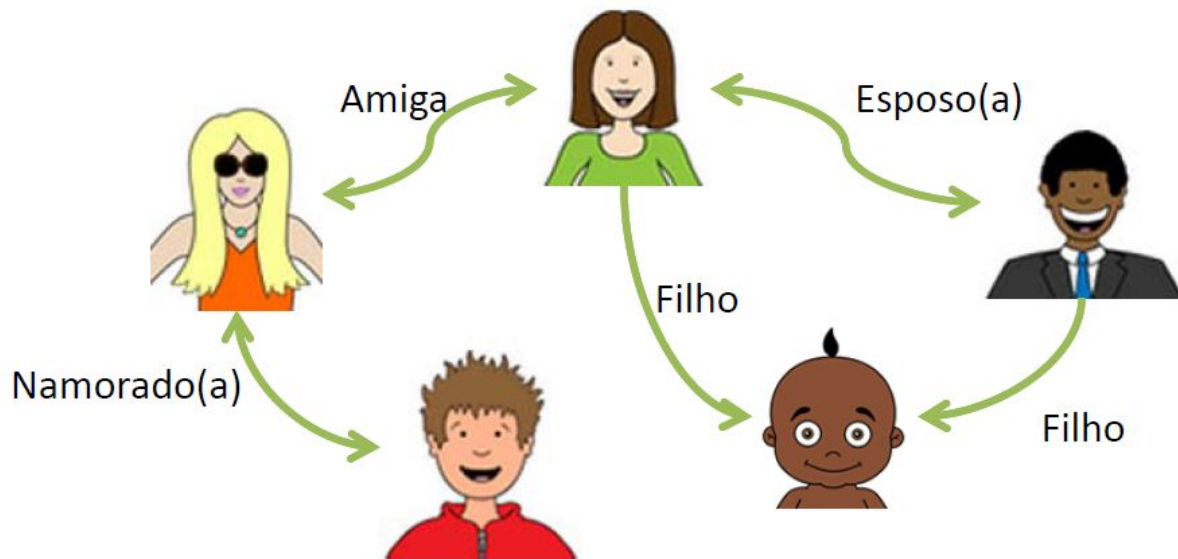
Grafos em jogos

Máquina de Estados Finitos



O que é um grafo?

Forma genérica de estruturar **informações**, relacionando-as através de **pares**



Conceitos relacionados a grafos

Um **Grafo** é um conjunto de:

Vértices (ou nós): representam informações

Arestas: representam as relações entre duas informações

$G = (V, A)$, onde

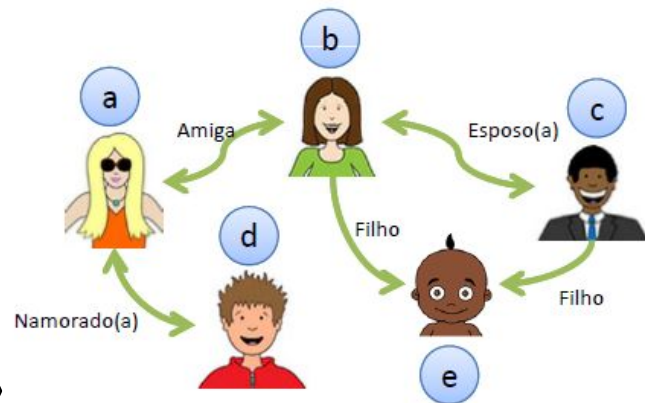
$V = \{v_1, v_2, \dots, v_n\}$

$A = \{a_1, a_2, \dots, a_m\}$, tal que $a_i = (u, v)$, onde u e $v \in V$

Exemplo:

$V = \{a, b, c, d, e\}$

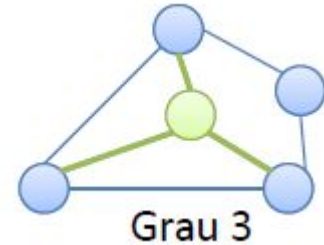
$A = \{(a,b), (b,c), (a,d), (b,e), (c,e)\}$



Conceitos

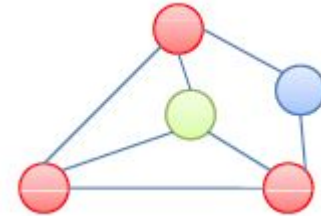
Arestas incidentes em um vértice

Grau de um vértice = número de arestas incidentes



Vértices adjacentes a um vértice

Dois vértices v e u são adjacentes se existe $(v, u) \in A$

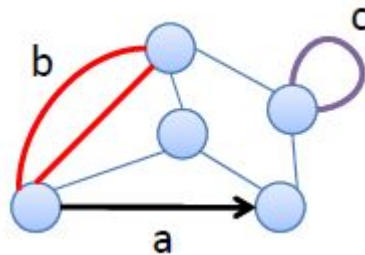


Tipos de arestas

Aresta dirigida (a)

Arestas paralelas (b)

Loop (c)



Conceitos

Caminho

Sequência de vértices $C = \{c_1, c_2, \dots, c_t\}$ tal que $C \subseteq V$ e $(c_{i-1}, c_i) \in A$ para $i > 1$

Sequência de arestas $D = \{d_1, d_2, \dots, d_{t-1}\}$ tal que $D \subseteq A$ e $\forall d_i = (u, v)$ para $i > 1, d_{i-1} = (x, u)$

Tamanho do caminho

Número de arestas que o caminho utiliza

Tipos de caminhos

Caminho (simples) = quando não há vértices / arestas repetidos

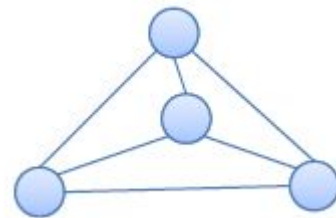
Ciclo = quando o vértice origem e destino são os mesmos

Ciclo (simples) = vértices/arestas não se repetem (exceto os de origem e destino)

Conceitos

Grafo completo

Para todo par de vértices $v, u \in V$ existe $(v, u) \in A$



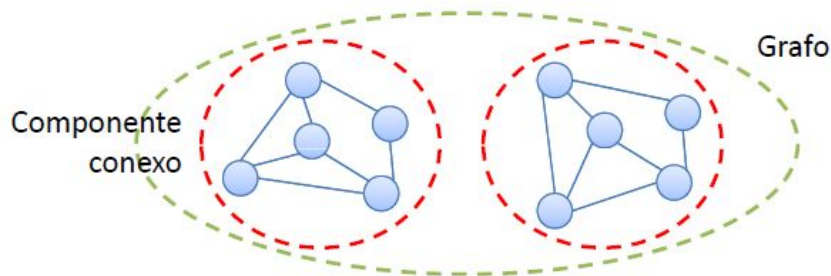
Subgrafo

O grafo $G' = \{V', A'\}$ é subgrafo de $G = \{V, A\}$ se $V' \subseteq V$ e $A' \subseteq A$

Componente conexo: Subgrafo $G' = \{V', A'\}$ onde

Existe um caminho conectando u e v , $\forall u, v \in V'$

Não existe caminho conectando u e v , se $u \in V'$ e $v \notin V'$

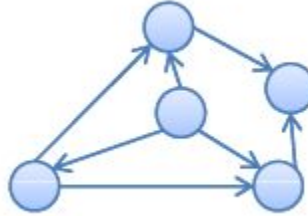


Grafo conexo = quando há apenas 1 componente conexo no grafo

Conceitos

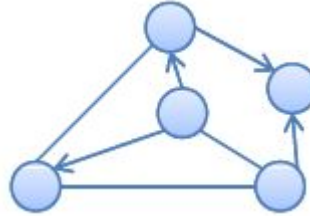
Grafo dirigido

Quando todas as arestas são dirigidas



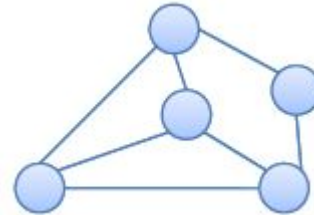
Grafo misto

Quando há arestas dirigidas e não-dirigidas



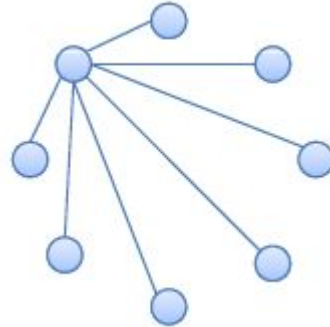
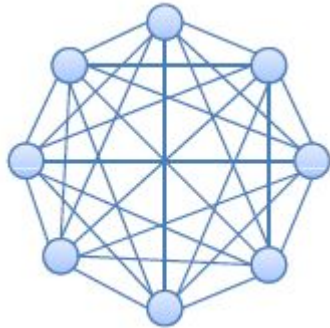
Grafo (simples)

Grafo conexo, não-dirigido, sem arestas paralelas ou loops



Relações entre vértices e arestas

Seja G um **grafo completo** com n vértices, o número de arestas de G é $O(n^2)$



Relações entre vértices e arestas

Seja G um grafo com n vértices e m arestas, a soma do grau de todos os vértices é $O(m)$.

Nessa soma, cada vértice contará suas arestas incidentes

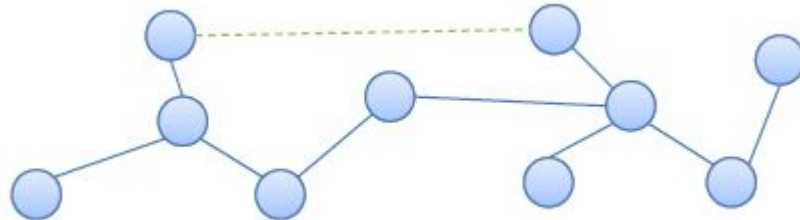
Cada aresta será contada duas vezes (vértices opostos) $\rightarrow 2m$

Seja T uma árvore com n vértices, o número de arestas é $n - 1$

1 aresta é suficiente para conectar um grafo com 2 vértices sem formar ciclos

A cada novo vértice no grafo, uma nova aresta seria necessária para conectá-lo aos demais

Portanto, se o grafo tiver n vértices, a árvore terá $n - 1$ arestas (uma aresta a mais forma ciclo)



Como representar um grafo

Por definição, um grafo é um conjunto de vértices e arestas.

Tipo abstrato de dado $\Rightarrow$ várias possibilidades de implementação

A estratégia *ideal* depende das operações que você fará sobre o grafo

- Consultas (quais são os adjacentes de um vértice? Existe uma aresta entre dois vértices? etc.

- Inserção de vértices/arestas

- Remoção de vértices/arestas

Algumas possíveis estruturas (não se limita a apenas estas formas)

- Matriz de adjacência

- Lista de arestas simples

- Lista de adjacência

Matriz de adjacência

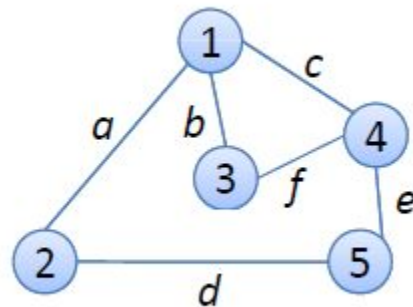
Grafo com n vértices representado por uma matriz $n \times n$

Os vértices são representados pelos índices da matriz

As arestas são representadas pelos conteúdos das células

Uma célula (i,j) “não vazia” representa uma aresta do vértice i ao j

	1	2	3	4	5
1	-	a	b	c	-
2	a	-	-	-	d
3	b	-	-	f	-
4	c	-	f	-	e
5	-	d	-	e	-

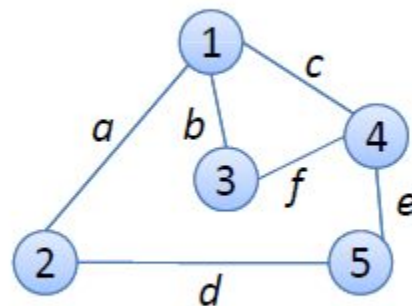


Lista de arestas (simples)

Forma intuitiva de definir grafos com inserção/remoção ao longo do tempo

Lista (ou array) de arestas com seus vértices

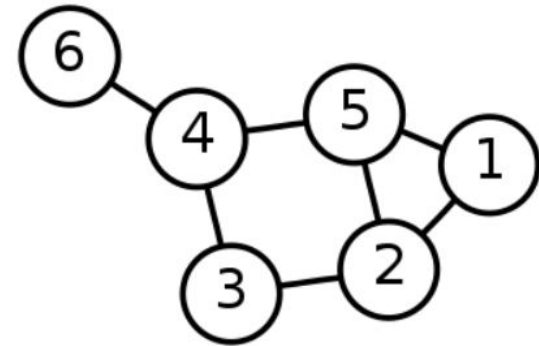
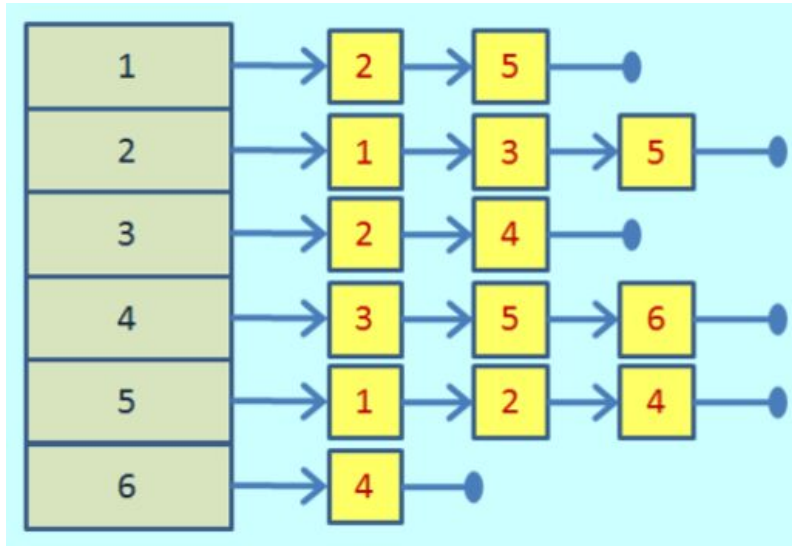
Aresta	v1	v2
<i>a</i>	1	2
<i>b</i>	1	3
<i>c</i>	1	4
<i>d</i>	2	5
<i>e</i>	4	5
<i>f</i>	3	4



Lista de adjacência

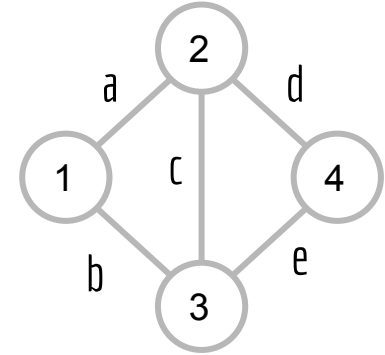
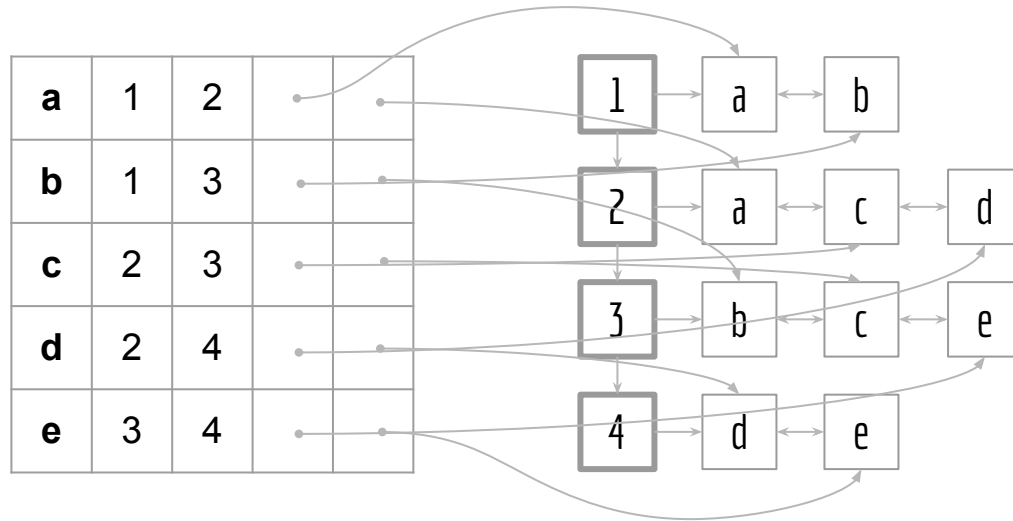
Cada vértice possui uma lista (ou array) para seus adjacentes

Melhora o desempenho dos percursos (a partir de um vértice, visitar seus adjacentes)



Estruturas mais complexas

A estrutura pode variar em função da necessidade da “velocidade” das operações sobre o grafo: operações mais rápidas requerem mais informações.

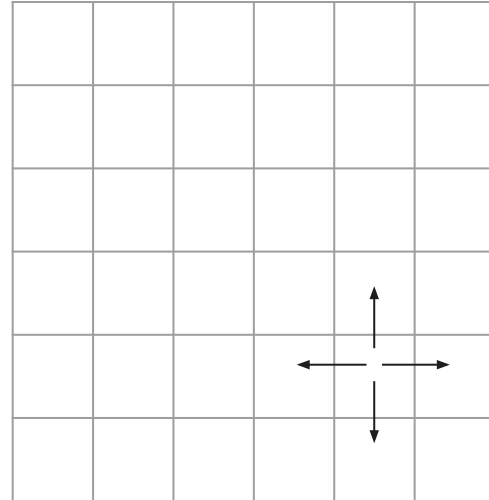


Estruturas bem mais simples

Em função do contexto, as operações podem ser triviais

Por exemplo, uma malha regular vista como um grafo

- Cada célula é um nó
- Células vizinhas possuem arestas entre si

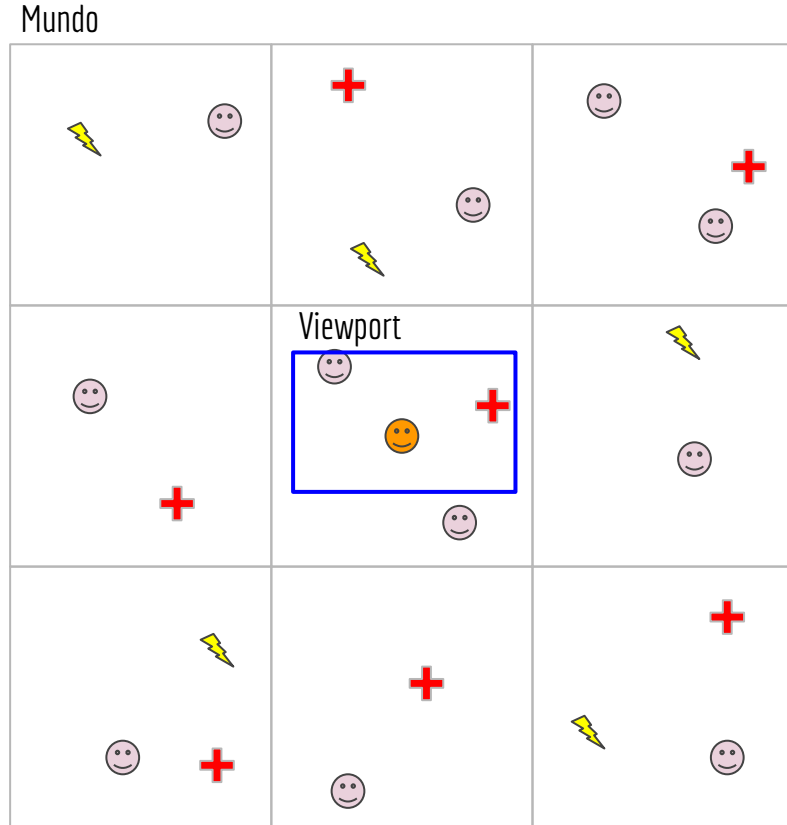


Atividade

Implementar um nível de jogo contendo 9 áreas organizadas em uma malha com:

- inimigos (NPCs)
- caixas de primeiros socorros
- caixas de munição

A janela de visualização (viewport) deve ser independente da malha



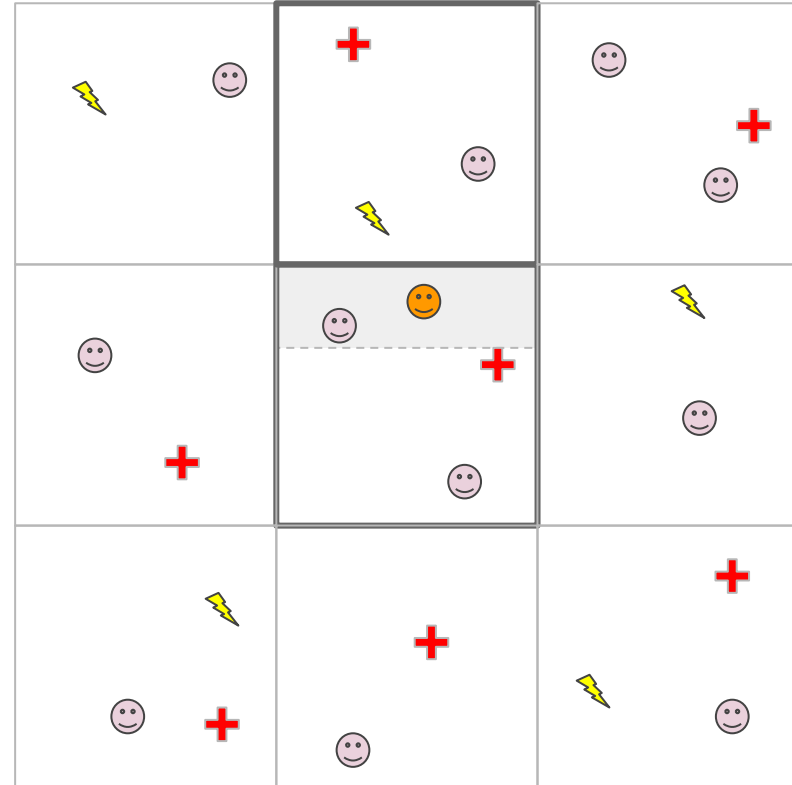
Atividade

Nem todas as áreas estarão ativas:

- Uma área vizinha à atual só ficará ativa quando o personagem chegar “próximo” a sua borda
- Haverão, no máximo, 4 áreas ativas

Apenas os NPCs das áreas ativas serão atualizados

- Ir em direção ao jogador
- Enquanto houver sobreposição com o jogador, haverá danos à saúde do jogador



Atividade

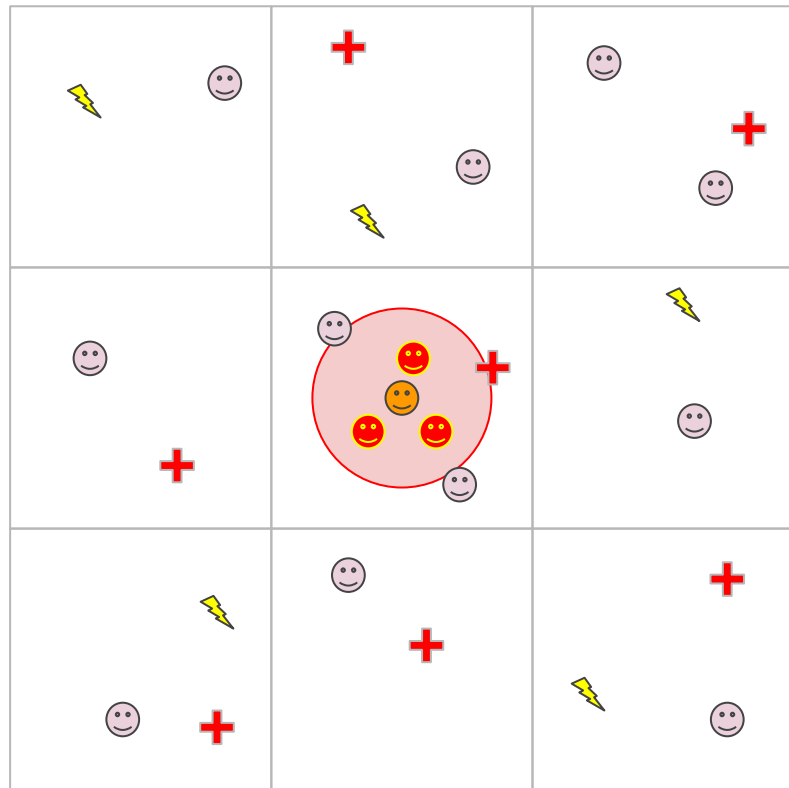
O jogador pode coletar itens de saúde (+) e munição (⚡):

- Ao usar +, a saúde do jogador é recuperada
- Ao usar ⚡, os inimigos ao redor sofrem danos

Qualquer personagem (NPC ou jogador) com saúde não positiva, morre

Objetivo do nível:

Sobreviver durante um determinado tempo



Atividade

O estado inicial do jogo pode ser:

- lido de um arquivo (cena, yaml, json...)
- gerado aleatoriamente

Teste o balanceamento em diferentes cenários:

- Dezenas/centenas/milhares de inimigos
- Áreas pequenas/médias/grandes
- Distância de ativação da área, dano, tempo, etc.

Bônus: Ter as áreas carregadas / liberadas dinamicamente da memória

